

Universidad Mariano Gálvez de Guatemala
Ingeniería En Sistemas de Información
Sede Chiquimulilla

Curso: ASEGURAMIENTO DE LA CALIDAD DE SOFTWARE
Docente: CARMELO ESTUARDO MAYÉN MONTERROSO

Tarea 4: Tipos de prueba

Estudiantes:

Sauly Estefania Cermeño Morales

Carnet:

179-21-8130



Introducción

El objetivo es explicar, de manera accesible y directa, lo crucial que son los tests de software y cómo contribuyen al desarrollo de apps de alta calidad. En un planeta crecientemente digitalizado, donde las aplicaciones son vitales para nuestro día a día, asegurar su correcto funcionamiento, velocidad y seguridad es algo más que un deseo: es una necesidad.

Los tests de software ayudan a detectar fallos, mejorar el desempeño y salvaguardar los datos antes de que el usuario final use el producto. Así, se confirma que la app cumple con lo prometido, ofrece una experiencia sin problemas y protege la integridad de los datos. Tests funcionales, que confirman que el sistema realiza las acciones y tiene las características previstas. Tests de rendimiento, centrados en evaluar la rapidez, solidez y respuesta en diversas circunstancias. Tests de seguridad, pensados para descubrir puntos débiles y evitar ataques que pongan en riesgo la información o el sistema tests son clave en todo el proceso de creación del software, desde el inicio hasta su lanzamiento, garantizando que la app funcione bien, de forma eficiente y segura.

Pruebas Funcionales

Las pruebas funcionales son un tipo de prueba de software que verifica la funcionalidad de un sistema o aplicación de software verificando que el sistema se comporte de acuerdo con los requisitos funcionales especificados y satisfaga las necesidades comerciales previstas.

El objetivo de las pruebas funcionales es validar las características, capacidades e interacciones del sistema con diferentes componentes. Implica probar la entrada y salida del software, la manipulación de datos, las interacciones del usuario y la respuesta del sistema a diversos escenarios y condiciones. Las pruebas funcionales sólo tienen como objetivo validar si un sistema funciona según lo previsto.

Beneficios de las pruebas funcionales

Estos son los principales beneficios de las pruebas funcionales:

Identifique los errores de forma temprana: Ayuda a identificar errores y fallos bastante temprano en el proceso de desarrollo.

Validación de requisitos: Ayuda a garantizar que la aplicación cumpla con los requisitos y especificaciones del usuario como se esperaba,

Mejor experiencia de usuario y calidad: Al garantizar una funcionalidad perfecta, las pruebas funcionales ayudan a mejorar la experiencia del usuario y la calidad y confiabilidad del software.

Cumplimiento normativo: Garantiza que el software cumpla con los estándares, regulaciones y legalidades de la industria.

Compatibilidad entre plataformas: Verifica que la aplicación pueda funcionar sin problemas en diferentes plataformas, entornos y condiciones de usuario.

Tipos de pruebas funcionales

Estos son los principales tipos de pruebas funcionales:

1. **Pruebas unitarias:** Esto lo realizan los desarrolladores que escriben scripts que prueban si los componentes/unidades individuales de una aplicación coinciden con los requisitos. Esto generalmente implica escribir pruebas que llamen a los métodos de cada unidad y los validen cuando devuelvan valores que coincidan con los requisitos.

En pruebas unitarias, cobertura de código es obligatorio. Asegúrese de que existan casos de prueba que cubran lo siguiente:

- Cobertura de línea
- Cobertura de ruta de código
- Cobertura del método

2. **Pruebas de humo:** Esto se hace después del lanzamiento de cada compilación para garantizar que la estabilidad del software esté intacta y no enfrente ninguna anomalía.

3. **Prueba de cordura:** Generalmente se realiza después de la prueba de humo y se ejecuta para verificar que cada funcionalidad principal de una aplicación esté funcionando perfectamente, tanto por sí sola como en combinación con otros elementos.

4. **Pruebas de regresión:** Esta prueba garantiza que los cambios en el código base código nuevo, estrategias de depuración, etc. no interrumpan las funciones ya existentes ni desencadenen cierta inestabilidad.

5. **Pruebas de integración:** Si un sistema requiere múltiples módulos funcionales para funcionar de manera efectiva, se realizan pruebas de integración para garantizar que los módulos individuales funcionen como se espera cuando

funcionan en combinación entre sí. Valida que el resultado de extremo a extremo del sistema cumple con estos estándares necesarios.

6. **Pruebas beta/de usabilidad:** En esta etapa, los clientes reales prueban el producto en un entorno de producción. Esta etapa es necesaria para evaluar qué tan cómodo se siente un cliente con la interfaz. Sus comentarios se toman para implementar mejoras adicionales al código.

7. **Prueba de caja blanca:** En las pruebas de caja blanca, el probador valida el funcionamiento interno de un sistema de software.

8. **Pruebas de caja gris:** Pruebas de caja gris Integra aspectos de las pruebas de caja negra y caja blanca, donde se prueban parcialmente las funciones internas de las aplicaciones. La funcionalidad y la seguridad se prueban aquí utilizando la arquitectura del sistema.

9. **Pruebas exploratorias:** En las pruebas exploratorias, los evaluadores exploran el software para detectar problemas y evaluar la experiencia del usuario sin depender de casos de prueba predefinidos.

10. **Pruebas de caja negra:** En las pruebas de caja negra, la funcionalidad del sistema de software se valida sin examinar sus funciones internas. Los evaluadores evalúan las entradas, salidas e interacciones de los usuarios para garantizar que los sistemas cumplan con los requisitos.

11. **Prueba de componentes:** Las pruebas de componentes se realizan inmediatamente después de las pruebas unitarias. En este enfoque, los objetos de prueba se prueban de forma independiente como un componente sin combinarse con otros componentes.

12. **Pruebas de bases de datos:** Las pruebas de bases de datos verifican la precisión y confiabilidad de los datos dentro de una base de datos asegurándose de que aspectos de la base de datos como el esquema, CRUD Crear, Leer, Actualizar, Eliminar, etc. funcionen como se espera en diferentes condiciones.

13. **Pruebas de recuperación:** Las pruebas de recuperación validan la capacidad de la aplicación para recuperarse de varios tipos de fallas.

14. **Pruebas estáticas:** Las pruebas estáticas son un tipo de enfoque de pruebas funcionales que incluye la evaluación del código, el diseño o la documentación del software sin ejecutar realmente el programa.

Principales herramientas y marcos de pruebas funcionales

Estas son algunas de las herramientas y marcos más confiables utilizados para las pruebas funcionales en 2025:

1. **Automatización de BrowserStack:** Plataforma basada en la nube para ejecutar pruebas funcionales automatizadas en 3,500+ navegadores y dispositivos reales.
2. **Automatización de aplicaciones BrowserStack:** Permite pruebas funcionales automatizadas de aplicaciones móviles en dispositivos iOS y Android reales.
3. **Selenium:** Herramienta de código abierto ampliamente utilizada para automatizar aplicaciones basadas en navegador en diferentes navegadores y plataformas.
4. **Ciprés:** Marco de pruebas rápido y fácil de usar para desarrolladores para aplicaciones web modernas. Funciona directamente en el navegador.
5. **Dramaturgo:** Framework de Microsoft para pruebas confiables de un extremo a otro de aplicaciones web en varios navegadores.
6. **Titiritero:** Node.js para automatizar Chrome y Chromium. Adecuado para pruebas sin cabeza y comprobaciones funcionales rápidas.
7. **Appium:** Marco de código abierto para automatizar aplicaciones web nativas, híbridas y móviles en iOS y Android.
8. **Cypress:** es una herramienta de código abierto utilizada para el testing de aplicaciones más modernas, utilizando el lenguaje de programación

JavaScript nos permite escribir diferentes tipos de pruebas de la pirámide de Cohn

9. **Selenium:** es una herramienta potente y popular para automatizar los navegadores web. Se usa ampliamente para probar aplicaciones web, automatizar tareas repetitivas y raspar sitios web.

Automatización de BrowserStack

Automatización de BrowserStack es una herramienta robusta diseñada para simplificar y mejorar la eficiencia de las pruebas automatizadas, lo que permite a los desarrolladores y Equipos de control de calidad para escalar sus pruebas y ofrecer funciones más rápido.

Esto elimina la necesidad de mantener una compleja infraestructura de pruebas interna, ahorrando tiempo y recursos. Garantiza que su producto funcione correctamente para una amplia audiencia al ofrecer acceso a dispositivos y navegadores reales, no solo simulaciones. BrowserStack proporciona una cobertura de prueba completa con más de 3,000 navegadores y 20,000 dispositivos iOS y Android reales.

Dado que 73% de los evaluadores utilizan pruebas automatizadas o secuencias de comandos para pruebas funcionales y de regresión AIMultiple Research, Está claro que la automatización ya no es opcional. Los desarrolladores y los equipos de control de calidad buscan soluciones que puedan ayudarlos a integrar las pruebas automatizadas sin problemas en sus flujos de trabajo.

Ventajas

1. Ofrece más de 20,000 dispositivos iOS y Android reales para pruebas funcionales, eliminando la necesidad de laboratorios físicos.
2. Permite ejecutar múltiples pruebas simultáneamente, reduciendo el tiempo de ejecución hasta 10 veces.
3. Se conecta fácilmente con Jenkins, GitHub Actions y otros para automatización en flujos DevOps.

4. Depuración avanzada incluye videos, capturas de pantalla y logs detallados para identificar errores funcionales rápidamente.
5. Soporte multi-plataforma compatible con diversas versiones de iOS y Android, asegurando pruebas cross-platform.

Desventajas

1. Los planes premium son caros, poco accesibles para startups o equipos pequeños.
2. Requiere conexión estable; interrupciones afectan las sesiones de prueba.
3. Usar BrowserStack Local para entornos internos puede ser difícil y lento.
4. Cuotas restringidas y falta de funciones avanzadas en la versión gratuita.
5. Algunos dispositivos, especialmente iOS, pueden tener tiempos de carga prolongados.

BrowserStack App Automate

Cómo se utiliza

BrowserStack App Automate se utiliza para pruebas funcionales automatizadas de aplicaciones móviles nativas, híbridas y web en dispositivos reales iOS y Android alojados en la nube. Los equipos escriben scripts de prueba usando frameworks como Appium o Espresso que simulan interacciones del usuario, como toques, deslizamientos o entradas de texto, y los ejecutan en múltiples dispositivos y versiones de sistemas operativos simultáneamente. La herramienta captura videos, capturas de pantalla y logs para depuración, y se integra con pipelines CI/CD para automatización continua. También permite pruebas en condiciones reales, como redes lentas o configuraciones de geolocalización.

Cuando se utiliza:

- **Fase de desarrollo:** Durante el desarrollo de aplicaciones móviles, para validar funciones específicas e.g., formularios, notificaciones push en dispositivos reales antes de pasar a integración.

- **Fase de integración:** En pipelines CI/CD e.g., Jenkins, GitHub Actions para ejecutar pruebas funcionales automáticas en cada commit, asegurando que nuevas funcionalidades no rompan flujos existentes.
- **Fase de pruebas:** Antes del lanzamiento, para pruebas de compatibilidad cross-device, verificando que la app funcione en diferentes modelos y versiones de iOS/Android.
- **Fase de mantenimiento:** Para pruebas de regresión, asegurando que actualizaciones no introduzcan errores en funcionalidades críticas.

Ejemplo práctico

Una empresa de e-commerce en 2025 usa BrowserStack para probar su aplicación móvil de compras en 50 combinaciones de dispositivos y sistemas operativos e.g., iPhone 14 con iOS 18, Samsung Galaxy S23 con Android 15. Los scripts automatizados verifican el flujo de checkout, desde agregar productos al carrito hasta procesar pagos, detectando un error de compatibilidad en Android 14 que causaba fallos en notificaciones push. La integración con GitHub Actions permitió ejecutar estas pruebas en cada pull request, reduciendo errores de compatibilidad en un 60% antes del lanzamiento.

Cypress

Cypress es una herramienta de código abierto utilizada para el testing de aplicaciones más modernas, utilizando el lenguaje de programación JavaScript nos permite escribir diferentes tipos de pruebas de la pirámide de Cohn. es su gran capacidad en aplicaciones modernas, si hablamos de las pruebas de interfaz de usuario desde el punto de vista de la aceptación del usuario, la velocidad con la que Cypress recorre los sitios web desarrollados en *frameworks* como Angular o React es increíble, al igual que en aplicaciones desarrolladas con otras tecnologías, llegando al punto que es casi imposible seguir la automatización con los ojos. la mayoría de estas aplicaciones web, que son desarrolladas con estos *frameworks*, nos dan la posibilidad de usar elementos dinámicos, lo cual complicaba tareas de

automatización con otras herramientas, con Cypress tenemos diferentes opciones para encontrar esos elementos que son todos unos loquillos.

En el ámbito de pruebas unitarias, para los conocedores de aserciones con Mocha y Chai, podremos volver a utilizar estas librerías, sumándole que, podremos diseñar nuestros test con BDD y TDD pudiendo depurar y ver con exactitud nuestra prueba en la herramienta de ejecución de Cypress y con una propia línea de comandos de la misma.

Así mismo construir pruebas para API's e integración de componentes, utilizando esta combinación de aserciones y procesos de desarrollo, añadiendo además utilidades como lo son *cucumber* para la escritura de pruebas BDD o Serenity BDD para nuestros reportes de pruebas con evidencias vivas. En cualquier caso, Cypress nos permite tener una cobertura de pruebas bastante amplia y así garantizar la calidad en todo el ciclo de vida del *software*.

Ventajas:

- Es una herramienta para realizar pruebas de extremo a extremo, pruebas de componentes, pruebas de integración y pruebas unitarias.
- Es fácil de instalar, tiene una curva de aprendizaje rápida, se registran las acciones realizadas y se puede ir hacia adelante y atrás de la ejecución de las pruebas (time travel), genera reportes concretos y su documentación es actualizada y amplia.
- Tiene “built-in-tools” que entrega inmediatamente todas las herramientas necesarias para automatizar, no hay que agregar drivers, nos permite utilizar varios navegadores como chrome, firefox, edge, y para ios se tiene la versión experimental que simula safari.
- Permite escribir pruebas “más rápidas, fáciles y confiables”.

Desventajas:

- No permite la automatización de aplicaciones mobile.
- No permite interactuar con más de 1 tab, en varias situaciones existen alternativas para mitigar la necesidad de usar varios tabs.

Cómo se utiliza

Cypress es un framework JavaScript diseñado para pruebas funcionales end-to-end E2E en aplicaciones web modernas. Los equipos escriben scripts en JavaScript o TypeScript para simular interacciones del usuario, como clics, entradas de texto, navegación y validaciones de UI, directamente en el navegador. Su arquitectura permite ejecutar pruebas en tiempo real, con una interfaz visual que muestra snapshots de cada paso, facilitando la depuración. Cypress captura automáticamente videos y capturas de pantalla en caso de fallos, y su dashboard cloud proporciona reportes detallados para equipos distribuidos. Se integra con herramientas CI/CD como GitHub Actions, Jenkins o CircleCI, permitiendo pruebas automatizadas en cada commit o pull request. Además, funciones como mocks y stubs permiten simular respuestas de APIs, ideal para probar aplicaciones frontend sin depender de backends reales.

Cuando se utiliza

Cypress se utiliza en las siguientes fases del ciclo de vida del desarrollo de software SDLC para pruebas funcionales end-to-end E2E en aplicaciones web modernas:

1. Durante la implementación de componentes frontend, para pruebas unitarias e integración de elementos de la UI, como formularios, botones o menús dinámicos en frameworks como React, Angular o Vue.
2. En pipelines CI/CD e.g., GitHub Actions, Jenkins, CircleCI, para pruebas automáticas de regresión e integración en cada commit o pull request.

3. Durante pruebas de sistema y E2E, para verificar flujos completos de usuario, como autenticación, checkout o navegación, y pruebas de accesibilidad o rendimiento client-side.
4. Antes de lanzamientos, para pruebas de regresión y compatibilidad cross-browser en navegadores como Chrome, Firefox o Edge.
5. Tras el lanzamiento, para pruebas de regresión que validen actualizaciones o parches en la UI sin introducir errores.

Pruebas de Rendimiento

Pruebas de rendimiento es un método de prueba de software que se utiliza para evaluar qué tan bien funciona una aplicación bajo cargas de trabajo, condiciones de red y volúmenes de datos esperados su objetivo principal es garantizar que la aplicación sea rápida, estable, escalable y responsiva en todos los entornos.

Las pruebas de rendimiento evalúan el rendimiento de un sitio web o una aplicación en determinadas condiciones. Se centra en factores clave como capacidad de respuesta, estabilidad, velocidad y confiabilidad para garantizar que el sistema funcione sin problemas son importantes porque garantizan tiempos de carga rápidos, reducen el riesgo de fallas e identifican posibles problemas de escalabilidad. Lanzar una aplicación sin realizar pruebas podría dañar la credibilidad y la reputación de una marca.

Importancia de las pruebas de rendimiento

Las pruebas de rendimiento son esenciales para garantizar que su sitio web o aplicación ofrezca una experiencia fluida y confiable para los usuarios. A continuación se presentan algunos beneficios de las pruebas de rendimiento.

- Identifica cuellos de botella en el rendimiento antes de que afecten a los usuarios.
- Garantiza tiempos de carga rápidos y una experiencia de usuario óptima, incluso con mucho tráfico.

- Ayuda a mejorar la escalabilidad del sistema para el crecimiento futuro.
- Detecta posibles puntos de falla y reduce los riesgos de tiempo de inactividad.
- Valida la estabilidad y confiabilidad de su sistema en diferentes entornos y condiciones.

Tipos de pruebas de rendimiento

1. Prueba de carga: Simula el tráfico de usuarios esperado para determinar cómo el sistema maneja el uso normal. Ayuda a identificar cuellos de botella y evaluar métricas de rendimiento como el tiempo de respuesta y el rendimiento.

2. Pruebas de estrés: Empuja el sistema más allá de su capacidad de carga normal para evaluar su punto de rotura. Esto ayuda a identificar cómo se comporta el sistema bajo tráfico extremo y con qué elegancia se recupera de una falla.

3. Prueba de picos: Prueba la capacidad del sistema para manejar aumentos repentinos y bruscos del tráfico. Comprueba si la aplicación puede gestionar picos de carga abruptos sin fallar ni ralentizarse drásticamente.

4. Pruebas de remojo/Pruebas de resistencia: Examina el rendimiento del sistema durante un período prolongado bajo una carga constante. Ayuda a descubrir fugas de memoria, degradación del rendimiento y otros problemas de confiabilidad a largo plazo.

5. Prueba de volumen: Se centra en el rendimiento del sistema al manejar grandes volúmenes de datos. Se utiliza para identificar problemas de rendimiento relacionados con el tamaño de la base de datos, la carga de archivos o el procesamiento de datos a escala.

6. Pruebas de escalabilidad: Evalúa la capacidad de la aplicación para escalar hacia arriba o hacia abajo según la demanda del usuario. Comprueba si el rendimiento se mantiene constante a medida que aumenta la carga de trabajo.

Las 20 mejores herramientas de pruebas de rendimiento en 2025

1. Apache JMeter

Apache JMeter es una poderosa herramienta de código abierto desarrollada por la Apache Software Foundation. Se utiliza ampliamente para pruebas de carga y rendimiento de aplicaciones web, API, bases de datos y otros sistemas basados en servidores. JMeter le permite simular varios escenarios y generar cargas para evaluar el rendimiento del sistema en diferentes condiciones.

2. Gatling

Gatling es una herramienta de prueba de carga de código abierto diseñada para ayudar a los evaluadores y desarrolladores a evaluar el rendimiento y la escalabilidad de las aplicaciones web. Es conocido por su utilización eficiente de recursos, alto rendimiento y capacidad para manejar grandes cargas simultáneas de usuarios.

Gatling se basa en los principios de E/S asíncrona y sin bloqueo, lo que le permite generar cargas elevadas sin abrumar los recursos del sistema.

3. Automatizar BrowserStack

Automatizar BrowserStack es una poderosa plataforma de pruebas que ahora ofrece pruebas de rendimiento de aplicaciones web integradas utilizando Lighthouse en combinación con la automatización de Playwright.

Permite a los desarrolladores y equipos de control de calidad evaluar el rendimiento de las aplicaciones web en navegadores y dispositivos reales y garantiza experiencias de usuario rápidas, estables y receptivas.

4. Medidor de brillo

BlazeMeter es una plataforma de pruebas de carga basada en la nube que permite a los desarrolladores y evaluadores realizar pruebas de carga y rendimiento de aplicaciones.

5. Langosta

Locust es una herramienta de prueba de carga distribuida, basada en Python y de código abierto que se utiliza para medir el rendimiento y la escalabilidad de aplicaciones web, API y otros servicios basados en red. Está diseñado para ser fácil de usar, altamente flexible y escalable, lo que lo convierte en una opción popular para pruebas de carga en diversos proyectos.

6. K6

K6 es una herramienta de pruebas de rendimiento, centrada en el desarrollador y de código abierto diseñada para pruebas de carga y pruebas de rendimiento de aplicaciones web y API. Permite a los desarrolladores y evaluadores medir el rendimiento y la escalabilidad de sus sistemas en diferentes condiciones de carga.

7. ApacheBench

ApacheBench (ab) es una herramienta de línea de comandos y un software de código abierto del proyecto del servidor HTTP Apache. Se utiliza para probar la carga de servidores web y aplicaciones midiendo su rendimiento y concurrencia en diferentes condiciones.

8. Tsung

Tsung es una herramienta de prueba de carga distribuida y de código abierto que se utiliza para probar el rendimiento y evaluar aplicaciones web, servicios web y otros protocolos de red. Desarrollado en Erlang, Tsung está diseñado para generar alta concurrencia y estrés en el sistema bajo prueba, permitiendo a los usuarios evaluar su rendimiento y escalabilidad.

9. Carga web

WebLOAD es una solución de pruebas de carga de alto rendimiento diseñada para aplicaciones web de nivel empresarial. Le permite simular el tráfico de usuarios a gran escala y medir el rendimiento de su aplicación bajo cargas pesadas. WebLOAD es conocido por su capacidad para crear escenarios de prueba realistas que reflejan fielmente las condiciones del mundo real, lo que lo hace ideal para pruebas de estrés y optimización del rendimiento.

10. Dynatrace

Dynatrace es una solución líder de gestión del rendimiento de aplicaciones (APM) que aprovecha la inteligencia artificial (IA) para proporcionar información profunda sobre el rendimiento de las aplicaciones, la experiencia del usuario y el estado de la infraestructura. Automatiza el proceso de monitoreo, lo que permite a los equipos identificar y resolver problemas de manera proactiva antes de que afecten a los usuarios.

Apache JMeter

Herramienta de código abierto basada en Java para pruebas de carga, estrés y rendimiento funcional. Soporta protocolos como HTTP/HTTPS, SOAP, REST, FTP y JDBC; incluye una interfaz gráfica (GUI) para crear planes de prueba, grabación de scripts, informes detallados en HTML y ejecución distribuida para simulaciones a gran escala. En 2025, integra con DSL de Java para scripting avanzado y es compatible con CI/CD.

El objetivo clave de JMeter es evaluar qué tan bien un sistema puede manejar una carga de trabajo específica, medir los tiempos de respuesta y el rendimiento, e identificar cuellos de botella en el rendimiento.

Características y capacidades clave:

Permite realizar pruebas distribuidas para la generación de carga realista en varias máquinas.

Admite una amplia gama de protocolos, incluidos HTTP, HTTPS, SOAP, JDBC y FTP, para realizar pruebas de rendimiento exhaustivas de diversas aplicaciones y API.

Amplias funciones de informes y análisis con representaciones gráficas para identificar cuellos de botella en el rendimiento.

La interfaz fácil de usar permite la fácil creación y personalización de scripts mediante GUI o secuencias de comandos con Groovy.

El ecosistema de complementos mejora la funcionalidad de JMeter con funciones avanzadas.

compatibilidad:

Compatible con Windows, macOS y Linux

Admite pruebas de servidores web populares como Apache, Nginx e IIS, así como bases de datos como MySQL, Oracle y PostgreSQL.

integración:

Ofrece una integración perfecta de CI/CD, lo que permite realizar pruebas de rendimiento dentro de los procesos automatizados de compilación e implementación.

Su integración con Selenium permite realizar pruebas exhaustivas, combinando evaluaciones funcionales y de rendimiento para aplicaciones web.

La compatibilidad con las herramientas de APM facilita la supervisión y el análisis de toda la pila de aplicaciones, proporcionando una visión holística del rendimiento.

Ventajas

1. Código abierto y gratuito, sin costos de licencia.
2. Interfaz gráfica amigable para configuración rápida sin codificación avanzada.
3. Soporte versátil para múltiples protocolos y tecnologías.
4. Escalabilidad para simular miles de usuarios concurrentes.
5. Integración fácil con pipelines CI/CD como Jenkins.
6. Análisis detallado bajo diferentes cargas.
7. Comunidad grande y plugins extensos.
8. Portable en cualquier sistema con Java.

Desventajas

1. Configuración manual para pruebas distribuidas en la nube.

2. Alto consumo de recursos en pruebas a gran escala.
3. Curva de aprendizaje para configuraciones complejas.
4. No simula navegadores reales (no ejecuta JavaScript nativamente).
5. Limitaciones en escalabilidad comparado con herramientas comerciales.
6. Puede requerir plugins adicionales para funcionalidades avanzadas.
7. Informes menos intuitivos sin personalización.
8. No ideal para principiantes sin conocimiento de Java.

Cómo se utiliza

JMeter se utiliza para crear planes de prueba mediante su interfaz gráfica, configurando hilos de usuarios virtuales para simular cargas en aplicaciones web, APIs, bases de datos o servicios HTTP, REST, SOAP, JDBC. Los scripts se graban o escriben manualmente, ejecutando pruebas de carga, estrés o rendimiento funcional, con análisis de métricas como tiempos de respuesta y throughput. Se integra con pipelines CI/CD y soporta ejecución distribuida para simular miles de usuarios.

Cuándo se utiliza

- **Fase de desarrollo:** Para pruebas unitarias de rendimiento en APIs o servicios, validando respuestas bajo cargas iniciales.
- **Fase de integración:** En pipelines CI/CD e.g., Jenkins para pruebas continuas de rendimiento tras cada commit.
- **Fase de pruebas:** Para pruebas de carga y estrés en aplicaciones completas, simulando escenarios reales como picos de tráfico.
- **Fase de despliegue:** Antes de lanzamientos, para validar estabilidad bajo cargas esperadas.
- **Fase de mantenimiento:** Para monitoreo continuo, asegurando que actualizaciones no degraden el rendimiento.

Gatling

Herramienta de código abierto basada en Scala para pruebas de carga de alto rendimiento, optimizada para web apps modernas. Soporta scripting en Scala, informes en tiempo real, simulación de usuarios virtuales y bajo consumo de recursos

Gatling se basa en los principios de E/S asíncrona y sin bloqueo, lo que le permite generar altas cargas sin abrumar los recursos del sistema.

Características y capacidades clave

Ofrece un DSL que simplifica la creación de scripts de prueba, lo que hace que los escenarios sean más legibles y fáciles de mantener.

Proporciona monitoreo e informes en tiempo real de métricas clave de rendimiento, como tiempos de respuesta y tasas de error para una rápida identificación de cuellos de botella..

Permite manejar de manera eficiente miles de usuarios simultáneos, lo que ayuda a una evaluación precisa del rendimiento bajo cargas pesadas.

Las funciones de simulación avanzadas, como los comportamientos personalizados y el ritmo, ayudan a crear escenarios de carga complejos y realistas.

La integración perfecta con IDE populares mejora el desarrollo de scripts de prueba, mejorando la productividad del probador.

compatibilidad

Optimizado para probar aplicaciones web de carga, compatible con protocolos HTTP, HTTPS y WebSocket para probar diversos sistemas basados en web.

Compatible con varios navegadores web y sistemas operativos, lo que permite a los evaluadores evaluar el rendimiento de las aplicaciones en diferentes entornos.

integración

Se integra con populares Herramientas de compilación como Maven y Gradle, simplificando la ejecución de pruebas y la incorporación del flujo de trabajo.

Integración perfecta con Herramientas de CI/CD como Jenkins y Bamboo permiten realizar pruebas de rendimiento automatizadas dentro de las canalizaciones de integración y entrega continuas.

Gatling admite la integración con herramientas APM, combinando los resultados de las pruebas de carga con datos de infraestructura de aplicaciones en tiempo real para un monitoreo integral del rendimiento.

Ventajas:

1. Alto rendimiento con bajo uso de recursos.
2. Informes detallados en tiempo real.
3. Optimizado para aplicaciones web modernas.
4. Código abierto y gratuito.
5. Scripting avanzado para personalización.
6. Eficiente para pruebas distribuidas.
7. Integración con CI/CD.
8. Soporte para escenarios complejos.

Desventajas:

1. Requiere conocimiento de Scala.
2. Sin GUI amigable como JMeter.
3. Soporte limitado de protocolos.
4. Comunidad más pequeña.
5. Integración CI/CD requiere herramientas extra.
6. Menos plugins disponibles.
7. Curva de aprendizaje para no-programadores.
8. Limitado para entornos legacy.

Cómo se utiliza

Gatling emplea scripts en Scala para definir escenarios de carga, simulando usuarios virtuales en aplicaciones web o APIs (principalmente HTTP/REST). Se configura mediante código, ejecutando pruebas de alto rendimiento con bajo

consumo de recursos, generando informes detallados en tiempo real. Se integra con herramientas CI/CD y es ideal para entornos cloud-native.

Cuándo se utiliza

- **Fase de desarrollo:** Para pruebas de rendimiento en microservicios o APIs, validando escalabilidad temprana.
- **Fase de integración:** En pipelines CI/CD e.g., GitLab CI para pruebas continuas tras cambios en el código.
- **Fase de pruebas:** Para pruebas de carga en aplicaciones web modernas, simulando usuarios concurrentes.
- **Fase de despliegue:** Antes de lanzamientos, para verificar rendimiento bajo condiciones reales.
- **Fase de mantenimiento:** Para monitoreo de aplicaciones, asegurando estabilidad tras actualizaciones.

Pruebas de Seguridad

Las pruebas de seguridad son un aspecto importante de las pruebas de software centradas en identificar y abordar vulnerabilidades de seguridad en una aplicación de software. Su objetivo es garantizar que el software esté seguro contra ataques maliciosos, acceso no autorizado y violaciones de datos.

Las pruebas de seguridad implican verificar el cumplimiento del software con los estándares de seguridad, evaluar las características y mecanismos de seguridad y realizar pruebas de penetración para identificar debilidades y vulnerabilidades que podrían ser explotadas por actores maliciosos.

El objetivo de las pruebas de seguridad es identificar riesgos de seguridad y ofrecer recomendaciones de remediación para mejorar la seguridad general de la aplicación de software. Los evaluadores simulan ataques para verificar los mecanismos de seguridad existentes y buscar nuevas vulnerabilidades.

¿Por qué son importantes las pruebas de seguridad?

Protege datos confidenciales: Las pruebas de seguridad ayudan a garantizar que la información confidencial y sensible esté protegida contra el acceso no autorizado, la divulgación o el robo.

Previene violaciones de seguridad: Al identificar vulnerabilidades y debilidades en el sistema, las pruebas de seguridad ayudan a prevenir violaciones de seguridad y acceso no autorizado a datos confidenciales.

Mantiene la confianza: Las pruebas de seguridad ayudan a mantener la confianza de los clientes y usuarios al demostrar que el sistema es seguro y su información está protegida.

Cumple con los requisitos de cumplimiento: Muchas industrias y organizaciones están sujetas a regulaciones que requieren medidas de seguridad específicas, y las pruebas de seguridad ayudan a garantizar el cumplimiento de estas regulaciones.

Mejora la confiabilidad del sistema: Las pruebas de seguridad pueden ayudar a identificar y resolver debilidades de seguridad que pueden causar fallas o fallos del sistema, mejorando la confiabilidad general del sistema.

Herramientas de pruebas de seguridad

1. Seguridad Astra

Basado en estándares como OWASP, NIST y SANS25, De Astra La herramienta automatizada de pruebas de seguridad de aplicaciones y el escáner patentado ejecutan más de 10,000+ pruebas para identificar vulnerabilidades nuevas, emergentes y existentes en varios tipos de activos.

ayudan a garantizar cero falsos positivos, y un panel amigable para CXO, con generación de casos de prueba impulsada por IA y chatbot, ofrece una experiencia de prueba de seguridad de aplicaciones fluida. Además, nuestra extensión inteligente ayuda a registrar los inicios de sesión para simplificar el escaneo de pantallas protegidas y roles de usuario

2. Proxy de ataque Zed (ZAP))

zapear, como una de las herramientas de prueba de seguridad de aplicaciones web de código abierto más populares, Zed Attack Proxy (ZAP) está diseñado de acuerdo con las pautas de OWASP para encontrar vulnerabilidades en aplicaciones web.

Es una herramienta ampliamente utilizada por los profesionales de la seguridad y puede realizar escaneos automatizados y pruebas manuales, lo que la hace versátil para varios casos de uso para sus necesidades de seguridad de aplicaciones web.

3. OpenVAS

OpenVAS (Open Vulnerability Assessment System) es una herramienta integral de pruebas de seguridad de aplicaciones que forma parte del marco Greenbone Vulnerability Management (GVM).

Es adecuado para entornos pequeños y grandes y está diseñado para escanear la infraestructura de red y las aplicaciones web en busca de problemas de seguridad y proporciona informes detallados y evaluaciones de vulnerabilidad.

4. Nikto

Software de evaluación de vulnerabilidades de código abierto Nikto

Nikto es un escáner de servidor web de código abierto que realiza pruebas exhaustivas contra servidores web para múltiples elementos, incluidos más de 6.700 archivos y programas potencialmente peligrosos. También comprueba si hay versiones obsoletas de los servidores y otros problemas.

Como una de las herramientas de seguridad de aplicaciones, se especializa en pruebas de penetración e identificación de vulnerabilidades de seguridad en servidores web.

El enfoque de Nikto en el escaneo de servidores web lo convierte en una herramienta valiosa para identificar software obsoleto y configuraciones incorrectas. Su naturaleza de código abierto y su enfoque especializado lo convierten en una buena opción para las organizaciones que buscan un escáner de servidor web dedicado.

5. Mapa SQL

Mapa SQL es una herramienta gratuita de seguridad de aplicaciones que ayuda a identificar posibles vulnerabilidades a la inyección SQL.

Cuenta con un mecanismo de prueba resistente que se adapta a varios ataques y es compatible con varios servidores de bases de datos, como MySQL, Microsoft Access, IBM DB2 y SQLite

La fortaleza de SQLmap radica en su capacidad para identificar vulnerabilidades de inyección SQL. Su disponibilidad de código abierto y sus exhaustivos mecanismos de prueba lo convierten en una herramienta crucial para las organizaciones preocupadas por este tipo específico de ataque.

6. Mend.io

mend.io herramienta de seguridad de aplicaciones

Mend.io es una plataforma de seguridad de aplicaciones nativa de IA diseñada para canalizaciones de desarrollo modernas. Combina análisis de código estático, detección de riesgos de código abierto y capacidades únicas para proteger el código generado por IA y proteger los componentes de IA (detección estática y equipo rojo).

Su actualizador de dependencias incorporado (Mend Renovate) y la corrección impulsada por IA ayudan a los equipos a corregir los riesgos críticos más rápido y reducir la exposición general de las aplicaciones.

Mend.io ofrece un enfoque unificado para la seguridad de las aplicaciones, lo que permite a los equipos proteger toda su base de código, desde el código personalizado hasta los componentes de IA integrados. Su diseño centrado en la automatización, la priorización inteligente y las integraciones centradas en el desarrollador lo convierten en una opción destacada para las empresas que buscan obtener una visión holística del riesgo de sus aplicaciones.

7.WASP ZAP (Zed Attack Proxy)

OWASP Zed Attack Proxy ZAP a veces denominado Zed Attack Proxy o OWASP ZAP es un código abierto Herramienta de pruebas de seguridad de aplicaciones entre desarrolladores de software, equipos de seguridad empresarial y evaluadores de penetración por igual. ZAP fue fundada en 2010 por Simon Bennetts. Desde entonces, ZAP ha crecido hasta convertirse en una de las herramientas de seguridad gratuitas más populares, un estándar de la industria y el escáner de seguridad de aplicaciones más utilizado.

Seguridad Astra

Astra Security es una empresa de evaluación de vulnerabilidades y pruebas de penetración que proporciona servicios de pruebas de seguridad las 24 horas del día para evaluar los activos orientados a Internet de la manera más rápida y eficiente posible para detectar vulnerabilidades.

Características Principales

Protege contra ataques comunes como inyección SQL, scripting entre sitios (XSS), inclusión de archivos, y otros exploits de vulnerabilidades web.

Realiza escaneos periódicos y bajo demanda para identificar vulnerabilidades y malware en el código del sitio web, la base de datos y los archivos.

Herramientas automatizadas y soporte experto para detectar y eliminar malware de sitios web infectados.

Ayuda a recuperar el sitio de listas negras de motores de búsqueda y proveedores de seguridad, y monitorea la reputación en línea.

Defiende contra intentos repetidos y automatizados de adivinar credenciales de acceso.

Ofrece auditorías de seguridad exhaustivas para identificar debilidades y proporcionar recomendaciones.

Compatible con diversas plataformas de CMS como WordPress, Joomla, Drupal, y frameworks personalizados.

Proporciona una interfaz fácil de usar para monitorear la seguridad y gestionar la configuración.

Ventajas

- Ofrece una suite completa de herramientas para proteger un sitio web de diversas amenazas.
- Su WAF es fácil de configurar y gestionar, no requiere conocimientos técnicos avanzados.
- Proporciona asistencia experta en la limpieza de sitios infectados.
- Ayuda a los sitios a recuperarse de listas negras y a mantener una buena reputación en línea.
- Generalmente ofrece un buen soporte para consultas y problemas de seguridad.

Desventajas

- Puede ser una solución costosa, especialmente para pequeñas empresas o sitios web personales con presupuestos limitados.
- Como cualquier WAF, ocasionalmente puede bloquear tráfico legítimo si las reglas de seguridad son demasiado estrictas.

- Al externalizar la seguridad, se depende de la eficacia y las actualizaciones del proveedor.
- protege contra vulnerabilidades, no sustituye la necesidad de escribir código seguro desde el inicio.

Aplicaciones Prácticas

Garantiza que los sitios web en vivo estén protegidos 24/7 contra ataques maliciosos.

Ayuda a cumplir con estándares de seguridad y regulaciones al proporcionar un marco de protección robusto.

Es fundamental para limpiar y restaurar sitios web que han sido hackeados o infectados con malware.

Permite a los propietarios de sitios web y administradores monitorear la postura de seguridad de sus activos en línea.

Minimiza el riesgo de robo de datos, interrupciones del servicio y daños a la reputación.

Mapa SQL

Las herramientas de mapeo SQL o herramientas de diseño de bases de datos son aplicaciones que permiten a los desarrolladores y administradores de bases de datos visualizar, diseñar y gestionar la estructura de una base de datos relacional.

Características Principales

Permiten crear diagramas entidad-relación (ERD) o diagramas de clases para representar la estructura de la base de datos tablas, columnas, relaciones, claves primarias y foráneas.

Capacidad de generar un diagrama visual a partir de una base de datos existente.

Permiten generar scripts SQL DDL - Data Definition Language para crear la base de datos a partir del diseño gráfico.

Soporte para diferentes niveles de modelado de datos

Creación automática de documentación sobre el esquema de la base de datos.

Algunas herramientas pueden comparar esquemas de bases de datos y generar scripts para sincronizar diferencias.

Compatibilidad con bases de datos como MySQL, PostgreSQL, SQL Server, Oracle, etc.

Ayudan a gestionar las modificaciones en el esquema de la base de datos a lo largo del tiempo.

Ventajas

- Proporcionan una representación gráfica clara de la estructura de la base de datos, lo que facilita la comprensión.
- Ayudan a identificar errores de diseño antes de la implementación, como inconsistencias en las relaciones o la normalización.
- Agilizan el proceso de diseño y desarrollo de bases de datos al automatizar la generación de scripts DDL.
- Facilitan la comunicación entre los miembros del equipo al proporcionar un lenguaje visual común para el diseño de la base de datos.
- Generan documentación que es crucial para el mantenimiento y la comprensión futura de la base de datos.

Desventajas

- Algunas herramientas avanzadas pueden tener una curva de aprendizaje inicial.
- Para bases de datos muy simples, el uso de una herramienta de mapeo puede ser excesivo.
- : No siempre pueden capturar todas las complejidades específicas de un motor de base de datos particular
- Las herramientas más potentes y profesionales suelen ser de pago.
- El modelo gráfico debe mantenerse actualizado con los cambios reales en la base de datos, lo que requiere disciplina.

Aplicaciones Prácticas

Las herramientas de mapeo SQL/diseño de bases de datos se utilizan en el proceso de desarrollo para:

Son esenciales en la etapa inicial del desarrollo para diseñar la estructura de la base de datos antes de escribir cualquier código.

Para entender la estructura de una base de datos heredada o desconocida.

Para generar automáticamente los scripts SQL necesarios para crear la base de datos en un entorno de desarrollo o producción.

Para crear y mantener actualizada la documentación del esquema de la base de datos, lo cual es vital para el mantenimiento y la incorporación de nuevos desarrolladores.

Ayudan a visualizar el impacto de los cambios en el esquema y a planificar migraciones de bases de datos.

Permiten a los arquitectos de datos, desarrolladores y administradores de bases de datos trabajar juntos en un modelo común.

Conclusiones

1. Las pruebas funcionales, de rendimiento y de seguridad son esenciales para asegurar que el software sea confiable, eficiente y seguro, cubriendo aspectos clave de su funcionamiento y estabilidad.
2. La elección de herramientas adecuadas influye directamente en la calidad de los resultados, optimizando tiempos y reduciendo costos en el desarrollo.
3. Integrar estas pruebas de manera continua en el ciclo de vida del software mejora la detección temprana de errores y fortalece la confianza del usuario final.

Recomendación

1. Seleccionar herramientas basadas en el tipo de prueba, el presupuesto y la compatibilidad con la tecnología utilizada en el proyecto.
2. Realizar las pruebas de forma iterativa en todas las fases del desarrollo, no solo al final, para prevenir fallos costosos.
3. Capacitar al equipo para maximizar el uso de las herramientas y garantizar que las pruebas se apliquen de manera efectiva y consistente.

Referencias bibliográficas

Backer, A. (2025, July 22). BrowserStack: Automate vs live - which one do you need? *Muuktest.com*. <https://muuktest.com/blog/browserstack-automate>

Bose, S. (2025a, April 28). *Performance Testing: Types, importance and best practices*. BrowserStack. <https://www.browserstack.com/guide/performance-testing>

Bose, S. (2025b, June 24). *Functional testing: Definition, types & examples*. BrowserStack. <https://www.browserstack.com/guide/functional-testing>

Chaverra, A. (2022, November 10). *Cypress: Una verdadera experiencia de automatización*. Ceiba Software. <https://www.ceiba.com.co/ceiba-blog/cypress-una-verdadera-experiencia-de-automatizacion/>

Jimenezh, F. (2024, February 14). *Introducción a Cypress: ventajas, desventajas*. Medium. <https://medium.com/somos-pragma/cypress-c8c985ef36b5>

Felice, S. (2025, April 28). *Top 20 performance testing tools in 2025*. BrowserStack. <https://www.browserstack.com/guide/performance-testing-tools>

Severns, R. (2025, March 6). OWASP ZAP: Open source app security testing. *StackHawk, Inc.* <https://www.stackhawk.com/blog/guide-to-zap-application-security-testing/>

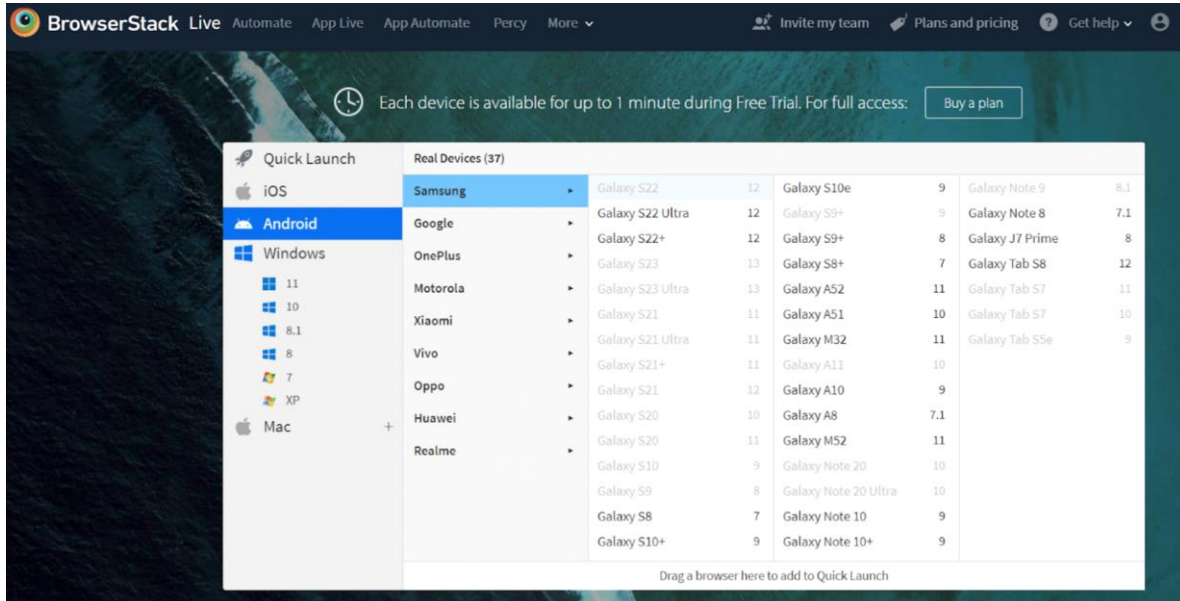
What Is Security Testing? (n.d.). HackerOne. Retrieved August 12, 2025, from <https://www.hackerone.com/knowledge-center/what-security-testing>

(N.d.). Getastra.com. Retrieved August 12, 2025, from <https://www.getastra.com/blog/security-audit/application-security-testing-tools/>

Anexos

BrowserStack

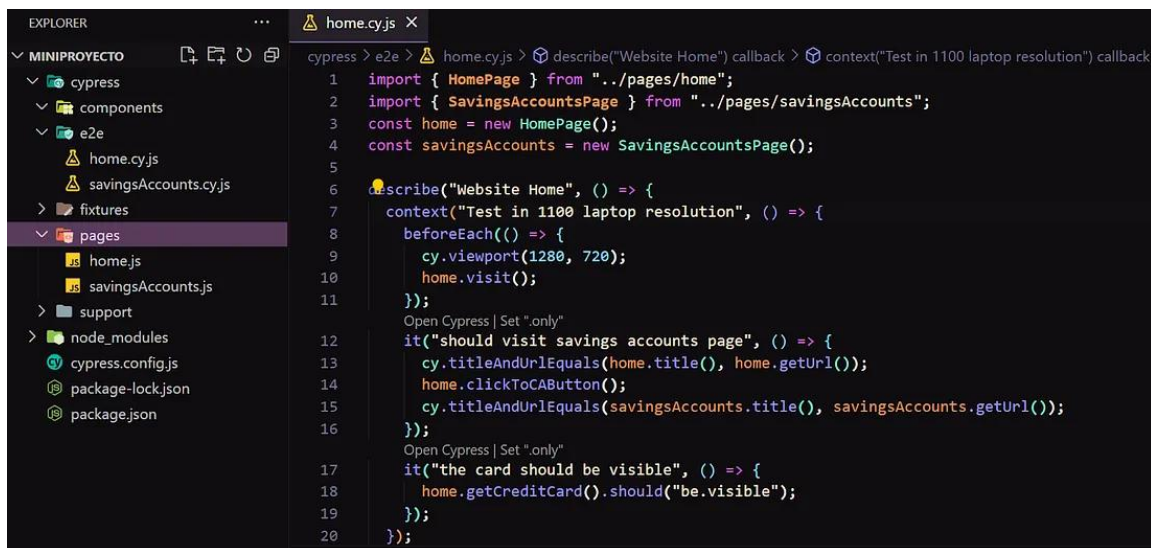
una plataforma de pruebas basada en la nube que permite a desarrolladores y equipos de QA realizar pruebas funcionales, de rendimiento y compatibilidad en aplicaciones web y



móviles

cypress

una herramienta de prueba front-end de próxima generación creada para la web moderna. Este framework de automatización de pruebas permite automatizar tanto pruebas a servicios como front-end



Gatling

Una herramienta de prueba de carga de código abierto diseñada para ayudar a los evaluadores y desarrolladores a evaluar el rendimiento y la escalabilidad de las aplicaciones web.



Proxy de ataque Zed

una de las herramientas de prueba de seguridad de aplicaciones web de código abierto más populares, Zed Attack Proxy (ZAP) está diseñado de acuerdo con las pautas de OWASP para encontrar vulnerabilidades en aplicaciones web

